

DWH + ETL

Why This Topic Is Critical

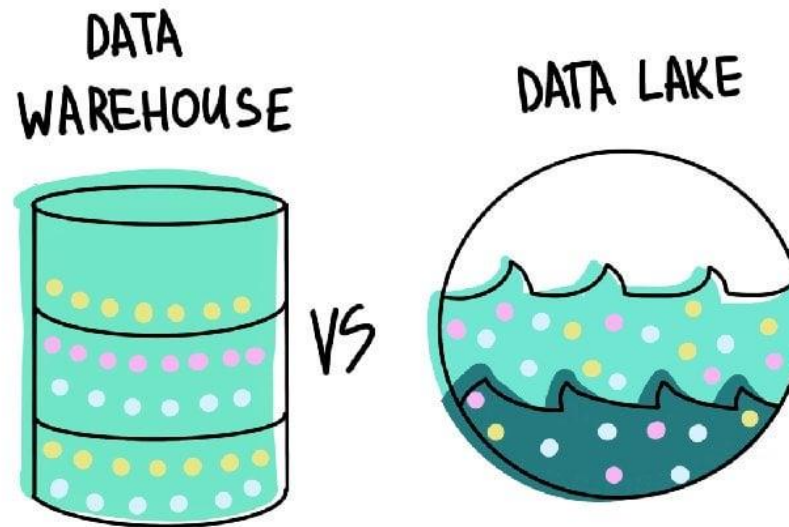
- Most BI outages begin in ingestion, not dashboards
- Small ingestion bugs create silent KPI drift
- Source systems change faster than analytics models
- Reliability requires explicit policies, not assumptions

Incident: Revenue Doubled After Rerun

- Nightly pipeline failed at 02:10 after loading 2 of 3 partitions
- Operator reran full job with plain `INSERT`
- Same transactions loaded again, revenue KPI jumped ~2x
- Root cause: no checkpoint + no idempotent target write

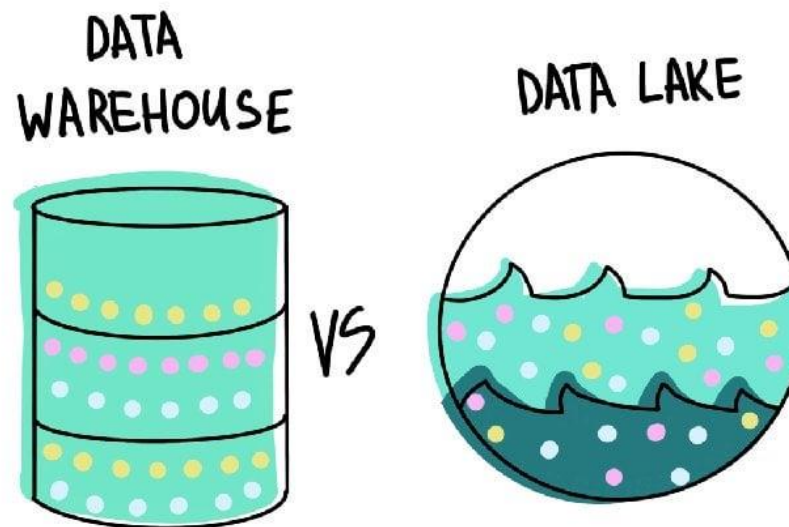
Core Stores: Warehouse vs Lake

- **Warehouse:** curated, schema-on-write, BI-first
- **Lake:** raw + processed zones, schema flexibility



Core Stores: Warehouse vs Lake

- Warehouse optimizes consistency and discoverability
- Lake optimizes ingestion agility and storage economics

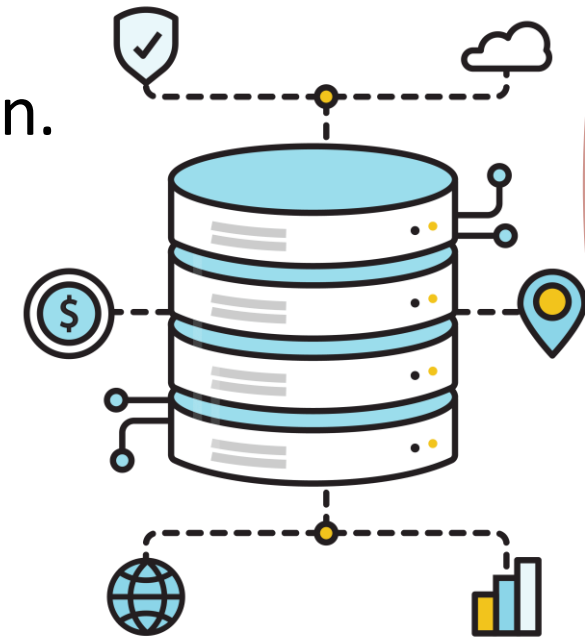


Data Warehouse Definition

- **Subject-oriented**
organized around major subjects (customer, product, sales)
- **Integrated**
multiple heterogeneous sources, cleaned and consistent
- **Time-variant**
historical perspective vs operational current-value data
- **Nonvolatile**
physically separate, operational updates do not occur in the warehouse

Why Separate Data Warehouse?

- Decision support requires historical data which operational DBs typically don't maintain.
- DS requires aggregation & summarization
- Different sources typically use inconsistent data representations



Schema-on-Write vs Schema-on-Read

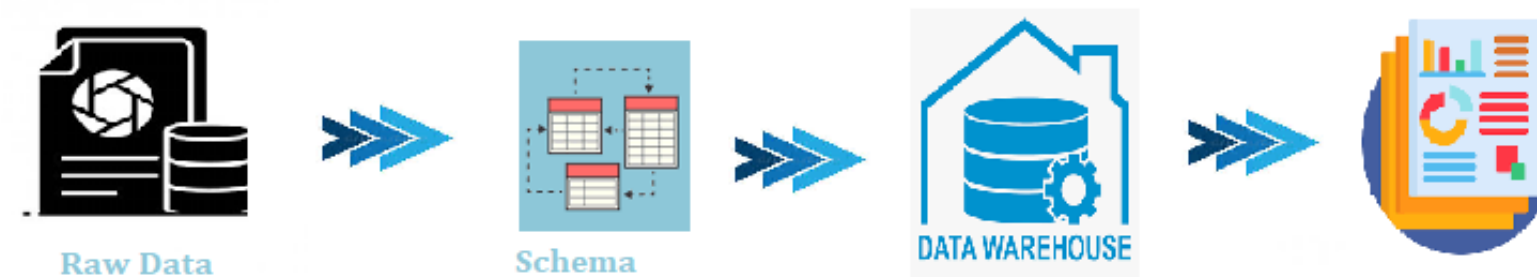
- Write-time schema validation catches issues earlier
- Read-time schema validation supports source variability
- Schema-on-write reduces BI surprise and metric drift
- Schema-on-read speeds onboarding but needs stronger governance

DWH vs Lake

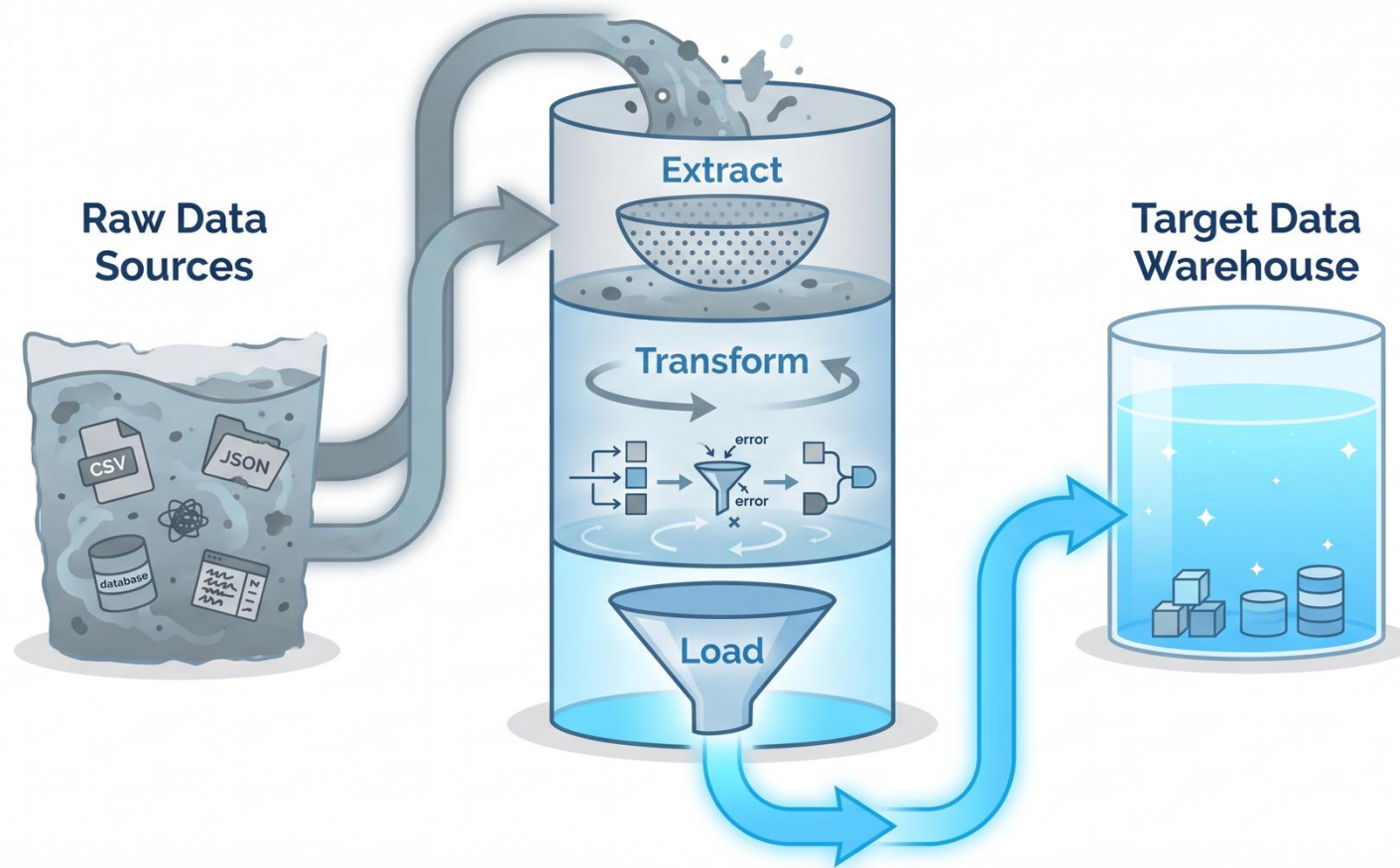
On Read



On Write



Extract, Transform, Load - ETL



Extraction

The main goal of extraction is to collect the data from the source system as fast as possible and less convenient for these source systems.



Transformation

All the collected data has been transformed into the same format

- **Filtering:** only relevant attributes going to DWH.
- **Cleaning:** Filling up the null values with specific default values.
- **Joining:** Join the multiple attributes into the one.
- **Splitting:** Splitting the single attribute into multiple attributes.
- **Sorting:** Sort the tuples based on the attributes.

Loading

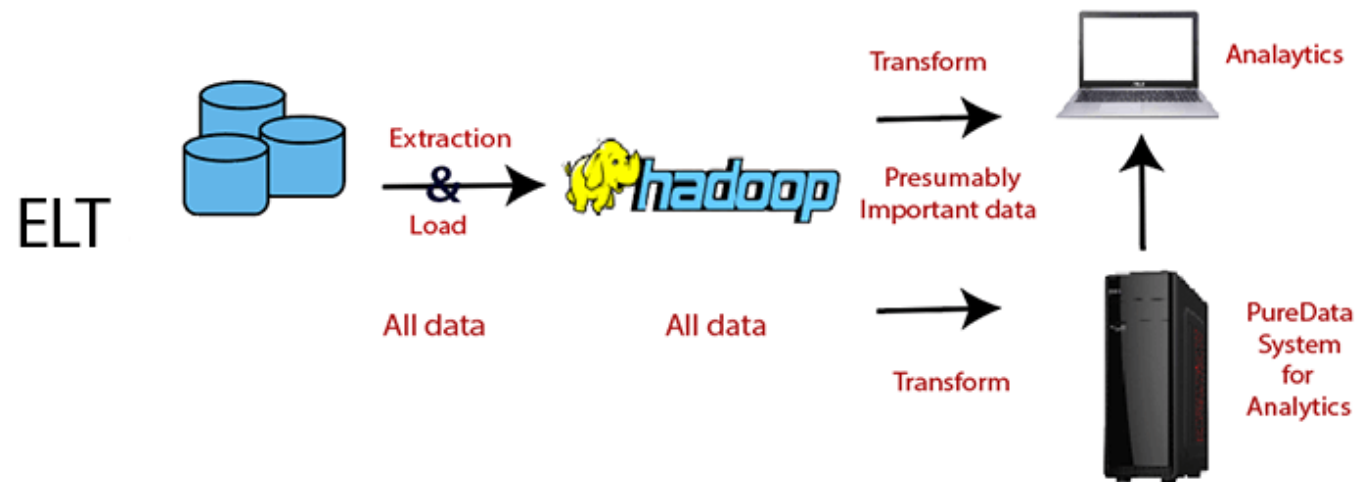
Loading is the final step of the ETL process.

The big chunk of data is collected from various sources,
transformed them,
loaded to the data warehouse.



Extract, Load, Transform - ELT

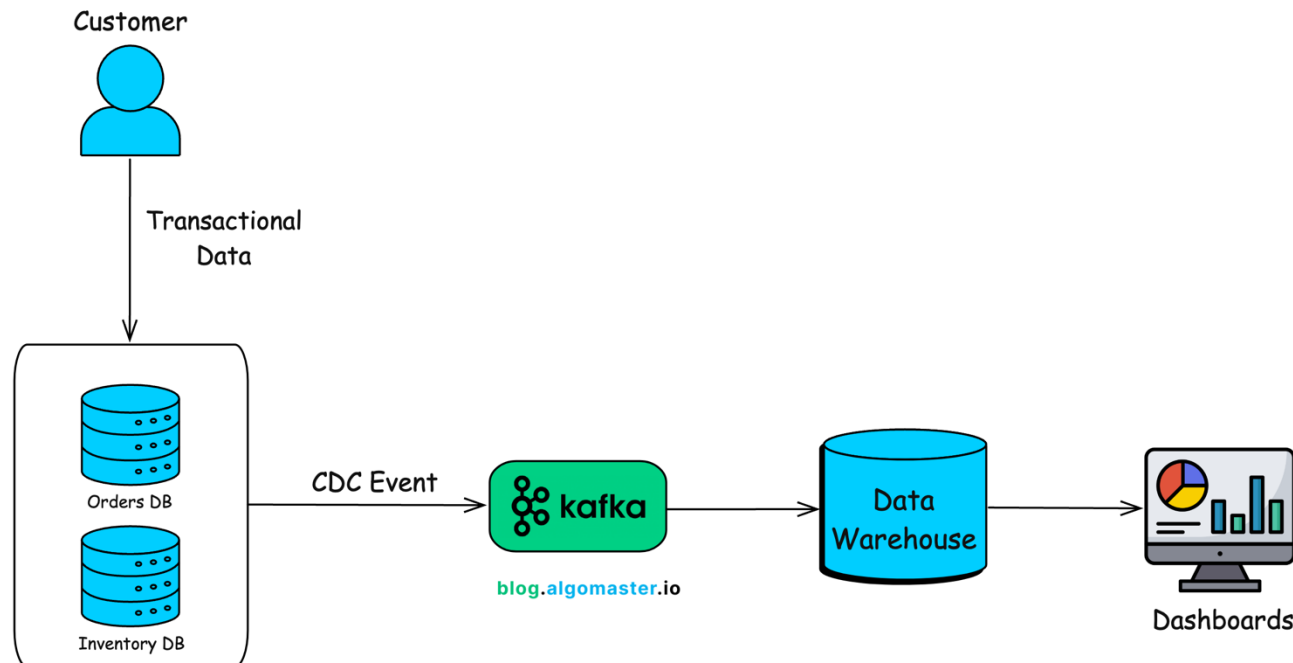
After extracting the data, we immediately start loading phase, moving all the data sources into a single, centralized data repository.



Today, most of the processes are ELT

Change-Data-Capture (CDC) Options

- **Full snapshot compare:** simple, expensive at scale
- **Timestamp:** easy, misses hard deletes unless modeled
- **Trigger-based CDC:** accurate but can burden source DB



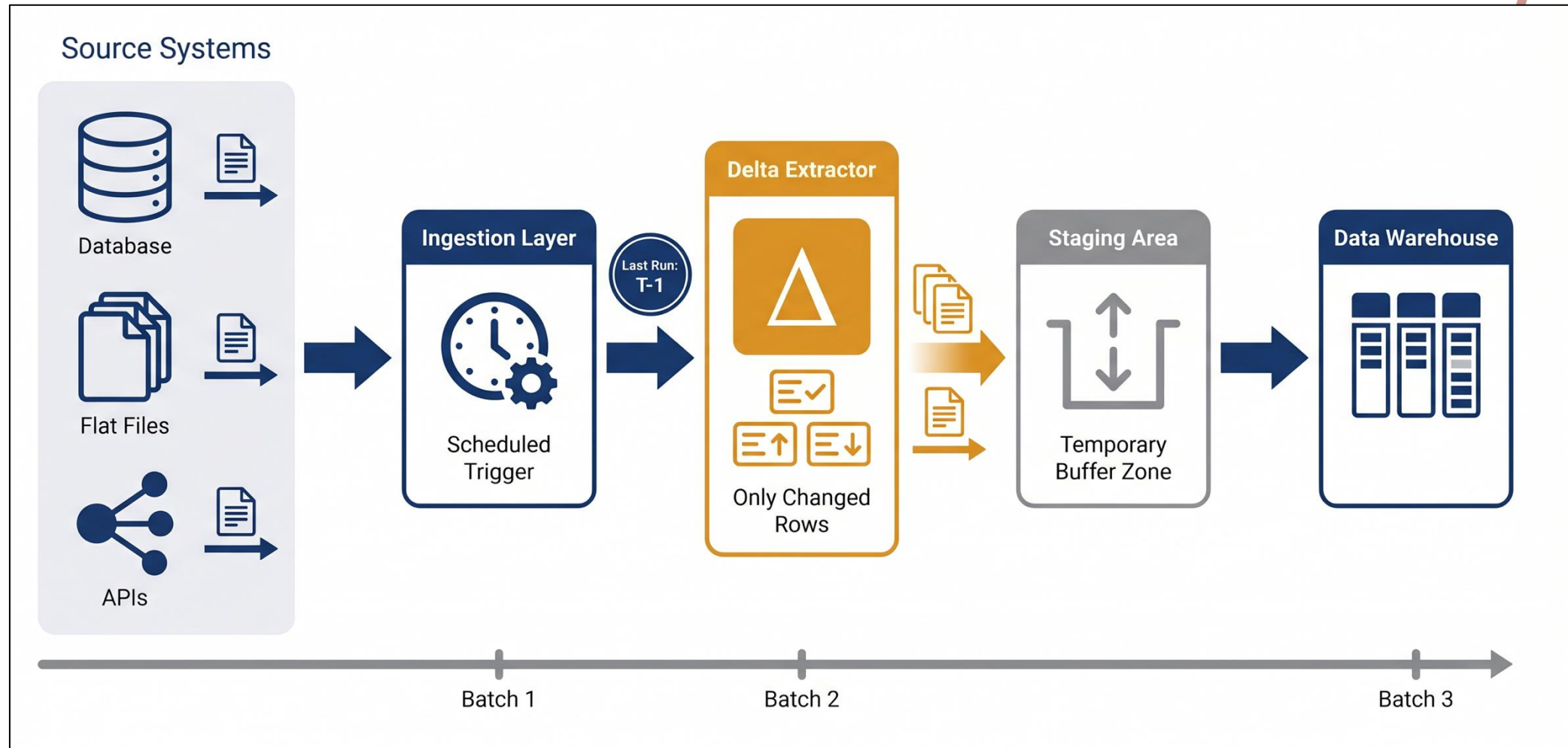
CDC Example: Orders Table

- **Inserts:** new orders must appear once
- **Updates:** status changes from `PENDING` to `PAID`
- **Deletes:** canceled rows may need tombstone semantics
- Policy must specify how each change type maps to analytics

Ingestion Modes

- Full refresh: easiest correctness, worst cost/latency at scale
- **Incremental batch: standard for most BI pipelines**
- Streaming: lowest latency, highest operational discipline

Ingestion Modes

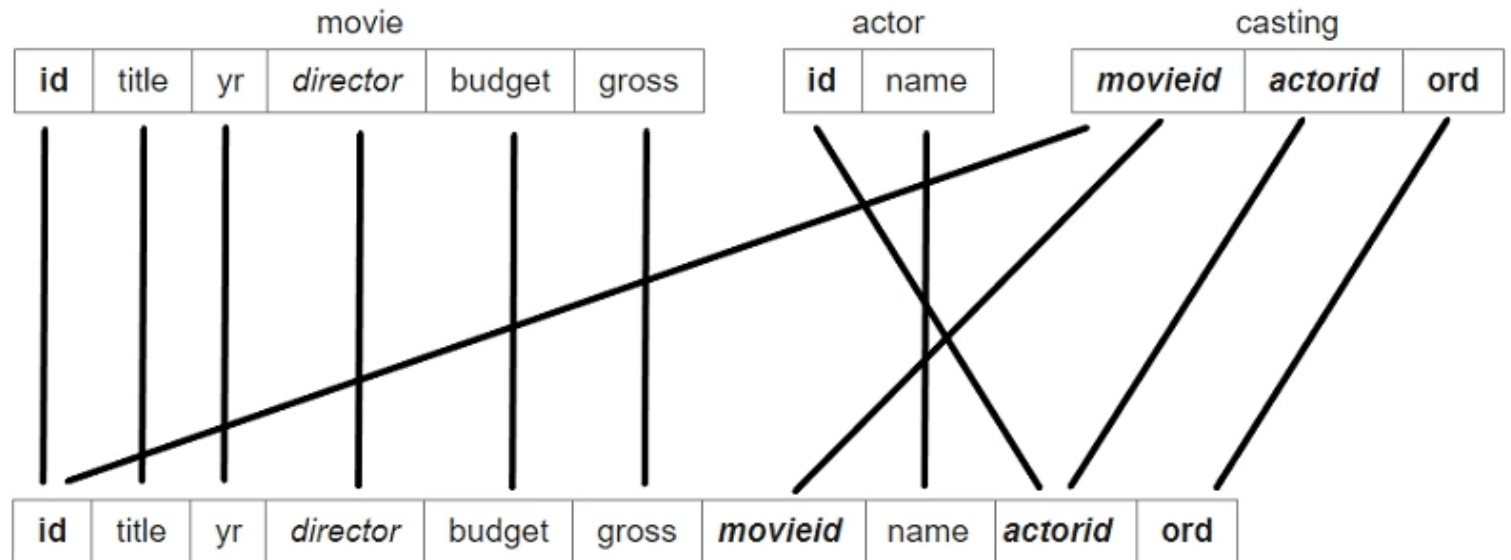


Source-to-Target Mapping (STTM)

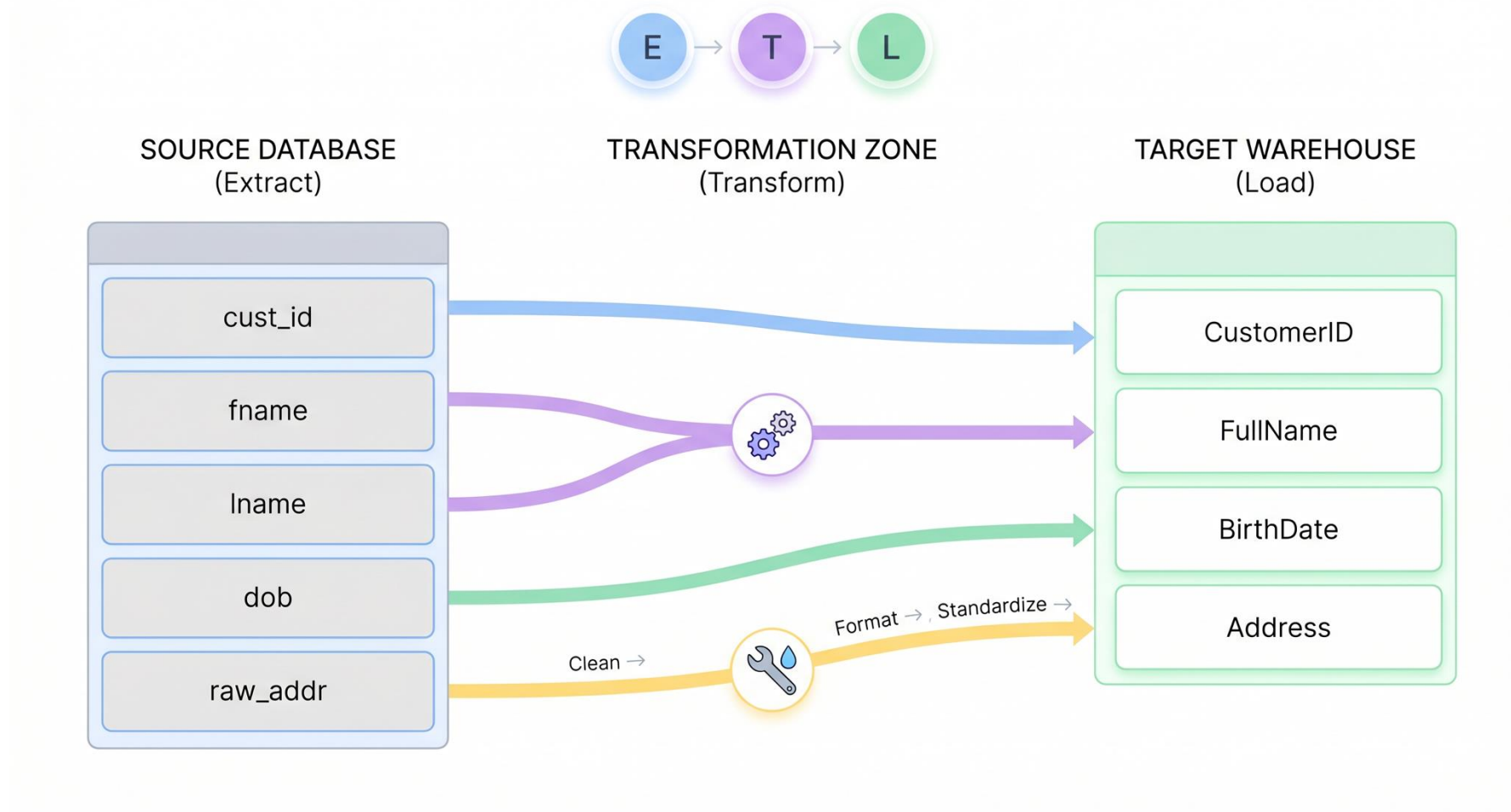
- Source and target schemas rarely match when moving data between systems
- a set of instructions defining how to transfer content.
- Critical when integrating multiple sources into WH.

STTM Mapping Flow

Information Sources



STTM Process Types



Idempotency Rule

$$f(f(D)) = f(D)$$

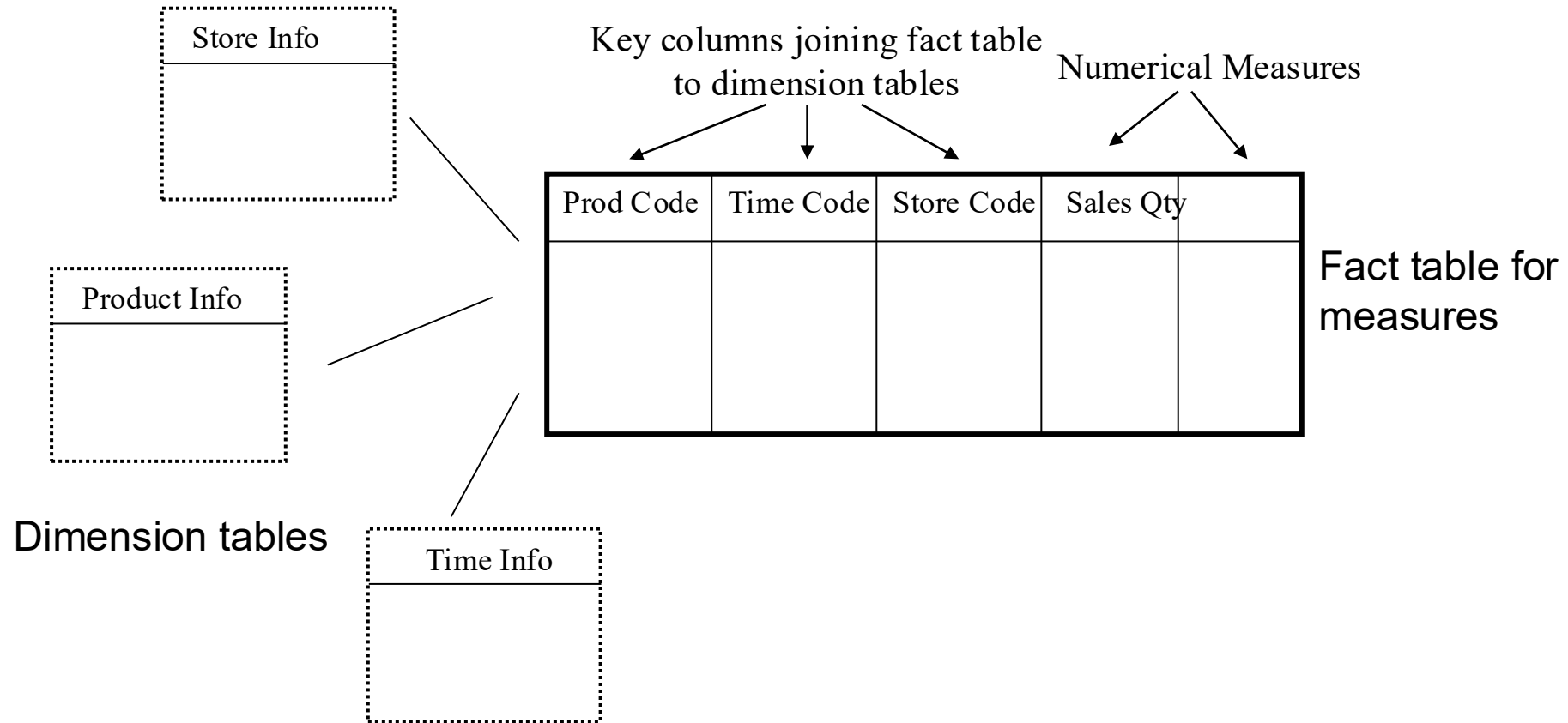
- Reprocessing same input **cannot** change final target state.
- Requires stable keys and **deterministic** transformation rules.
- Implement with conflict-safe MERGE/upsert patterns

DWH Schema Design

Fact and Dimension Contracts

- Fact table: measurable business events at one declared grain
- Dimension table: descriptive context used to filter, group, and explain facts
- Measure: numeric value with explicit aggregation semantics
- Hierarchy: governed drill path such as day -> month -> quarter -> year

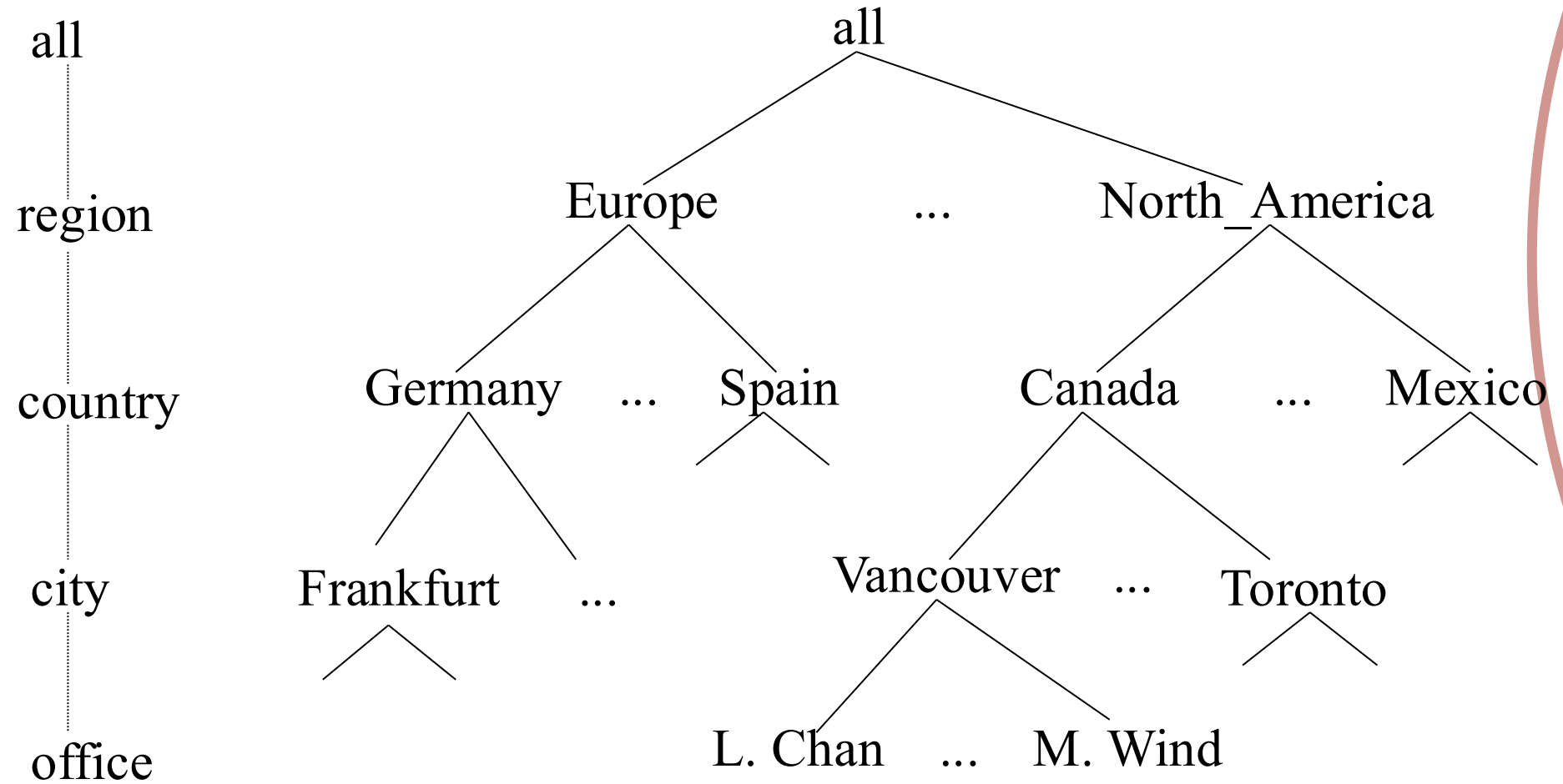
The Multi-Dimensional Model



Dimensional Modeling

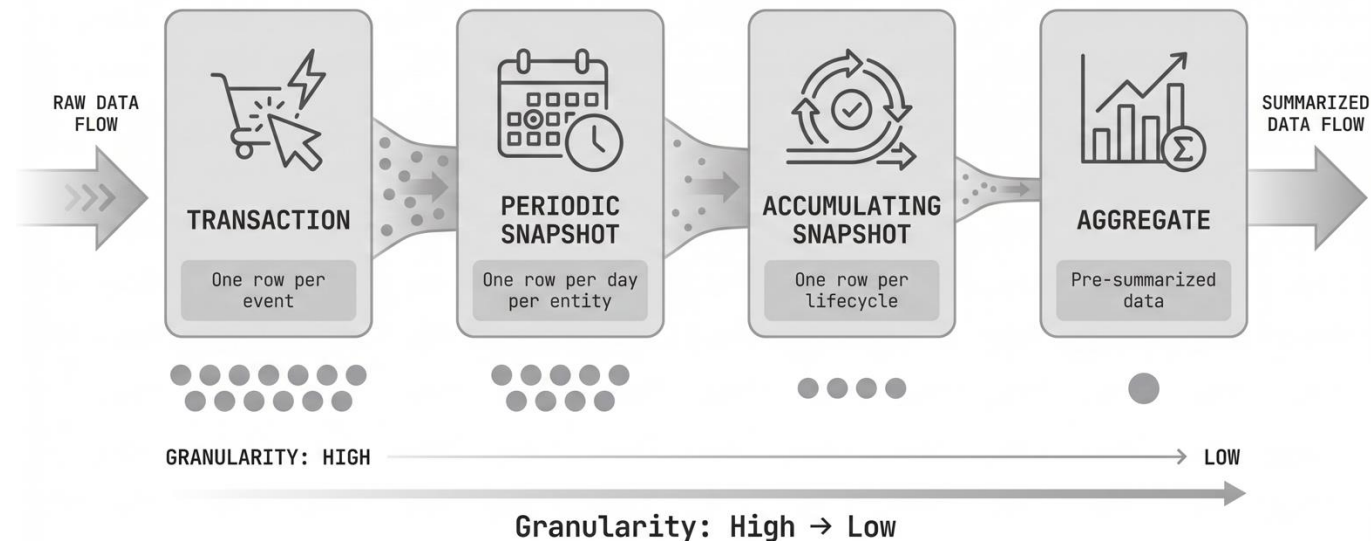
- Dimensions organize attributes into stable analysis paths
- E.g., Time: day -> week -> month -> quarter -> year
- E.g., Product: SKU -> product line -> brand -> category
- Conformed dimensions reuse the same keys and labels across facts
- Role-playing dimensions reuse one table in different roles, e.g., order_date and ship_date

Concept Hierarchy: Location Dimension



Declaring Fact Grain

- Grain defines exactly what one row in the fact table represents
- **If the grain is ambiguous, double counting is likely**
- Document grain and natural uniqueness before writing ETL



Grain: What It Is

- The grain is the business event level captured by the fact table
- It answers the question: what does one row mean?
- Changing grain later usually breaks joins, aggregates, and history

Common Grain Options

- Transaction grain: one row per event
- Periodic snapshot grain: one row per entity per period
- Accumulating snapshot grain: one row per process lifecycle
- Aggregate grain: pre-summarized rows
- Factless grain: event existence only

Choosing the Right Grain

- Start with the lowest atomic grain needed for analysis
- Use aggregate grains only for performance measures
- Avoid mixing grains in one fact table
- Check that the proposed grain has a stable unique key
- Write the grain statement before designing dimensions

Table Behavior Types

- Append-only
 - tables only insert new rows and preserve full event history
- Mutable tables
 - allow updates and deletes when the latest state matters
- Late-arriving
 - data reaches after the business event happened

Append-Only vs. Mutable Tables

- Append-only
 - best for logs, transactions, CDC history, and replayability
- Mutable
 - best for current-state dimensions, operational mirrors

A common pattern is append-only raw layer
plus curated mutable serving tables

Measure Types and Aggregation Rules

- Additive: safe to sum across all dimensions
- Semi-additive: sum across product, not across time
- Non-additive: calculate from numerator and denominator

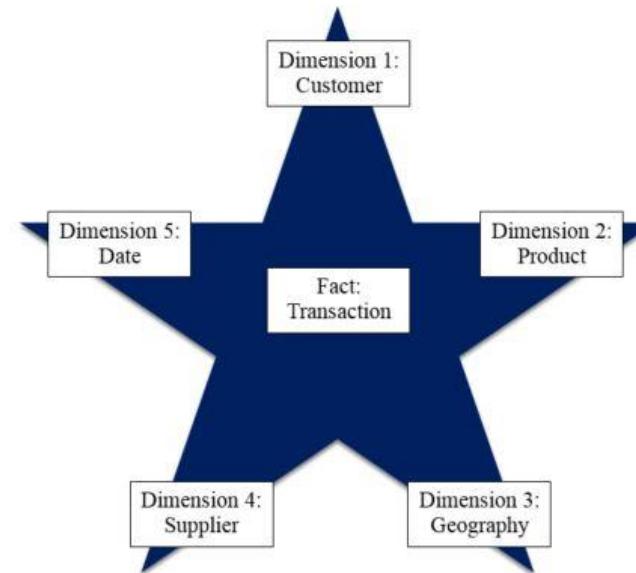
Common Schemas

Star Schema: Default BI Design



The Classic Star Schema

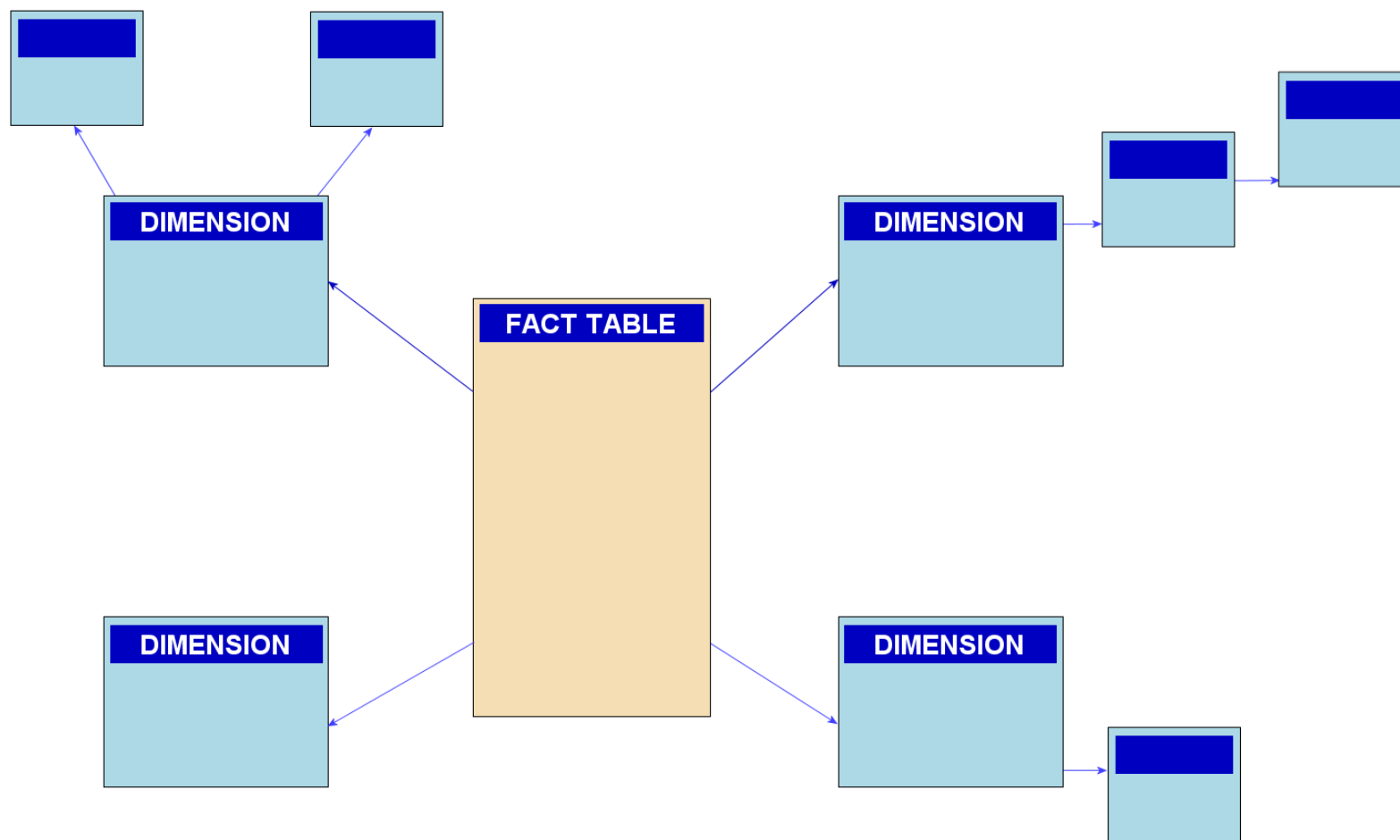
- A fact table sits at the center and connects to denormalized dimensions.
- Each dimension key appears once in the fact table primary/unique key.



Snowflake Schema

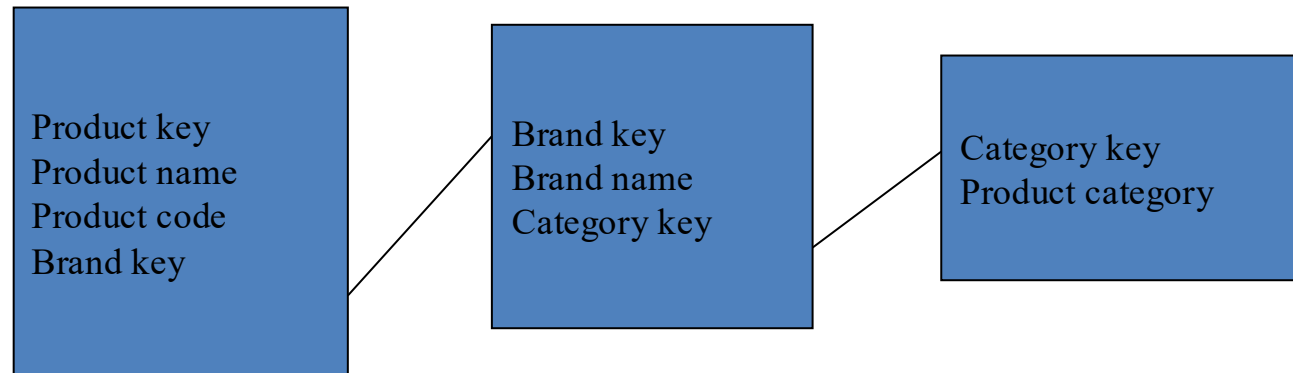
- Snowflaking normalizes selected dimension attributes into sub-dimensions
- It can reduce redundancy in large dimensions
- It adds joins, ETL complexity, and harder self-service BI
- Common compromise: normalized core plus star marts

Snowflake Schema: Normalized Dimensions



Snowflaking a Product Dimension

- Split shared hierarchy attributes into reference tables
- Product stores brand_key and category_key
- Benefit: less repetition and stronger governance
- Cost: extra joins unless a mart denormalizes back



Partitioning and Clustering Strategy

- Partition large facts by the dominant time filter, usually date_key
- Align partition granularity with query patterns and retention policy
- Cluster or sort on common filters such as customer_key or product_key
- Avoid tiny partitions and small files; they add metadata overhead
- Validate design with bytes scanned, partitions read, and p95 latency

Partition Pruning Cost Model

$$\text{ScanCost} = s \cdot |F|$$

$|F|$: total fact size

s : selected partition fraction

Example: $|F| = 1$ TB, one-day filter over 365 days ->

$s = 1/365$ - Scan drops from 1 TB to about 2.74 GB,

Cost and runtime often drop proportionally

Join Cost Intuition

$$\text{JoinWork} \approx O(|F_{\text{pruned}}| + \sum |D_i|)$$

Pruned fact size is usually dominant term

Broadcast joins help when dimensions fit memory

Large dimensions force shuffle and spill